

“Hate Speech and Toxic Comment Classification on Social Media Using Deep Learning”

Abstract: With the high growth of social media websites, communication all over the world has changed as people can contribute their opinions, ideas and information in real time and anywhere. Nonetheless, this development has also caused the harmful content to increase tremendously due to hate speech, abuse and toxic remarks. Such negative interactions may negatively influence people both psychologically and socially, so it is vital to create automated content moderation systems. The given project involves a deep learning system to identify and categorize hateful remarks on social media. The system adopts Natural Language Processing (NLP) with deep learning implementation, which are Convolutional Neural Network (CNN) and BiLSTM networks. The training and evaluation data was based on a large dataset of 159,571 comments with labels in “six toxicity levels: toxic, severe toxic, obscene, threat, insult, and identity hate”. Text cleaning, sequence padding and tokenizing data preprocessing methods were used to bring consistency and enhance model performance.

Local textual patterns were captured with the CNN model and the BiLSTM model was applied to comprehend the contextual relationships within text sequences. Hyperparameter optimization has been used to find the best learning rate, dropout rate and filter sizes to enhance the predictive performance of the model. “Evaluations of the models were conducted in terms of several performance measures, such as accuracy, precision, recall, F1-score, ROC-AUC, and Hamming loss”.

Moreover, a Gradio web interface was used to deploy the system and predict toxicity in real-time given the input by the user. The findings show that the deep learning models have the capability to detect dangerous content and assist the automated moderation systems. This project indicates the significance of AI-based solutions in developing safer online spaces and proposes further enhancement of the project, including

transformer-powered models and multilingual features.

Keywords: *Hate Speech Detection, Toxic Comment Classification, Deep Learning, CNN, BiLSTM, Natural Language Processing, Multi-label Classification, Social Media Analysis, Gradio Deployment, Text Classification*

1. BACKGROUND AND PROBLEM DEFINATION

Twitter, Facebook, and Reddit are examples of social media that have become indispensable communication and information sharing platforms. Although these sites have several positive aspects, they also expose their users to malicious information, such as hate speech, abusive remarks, and hate speech. This kind of content does not only impact on the individual users, but also interferes with the people online communities and entails unsafe digital space [1]. Content moderation is slow and ineffective when done manually because of the huge amount of information that is created with each second. Thus, artificial intelligence-driven automated systems are required to be effective in detecting and filtering toxic content. The field of deep learning, especially Natural Language Processing (NLP) has shown much potential in comprehending and categorizing textual information.

The project will concentrate on creation of a deep learning-based system to identify toxic comments. It is oriented to categorizing text in various types of toxicity with the help of sophisticated machine learning methods. The system uses both “Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM)” to extract both local and contextual features of text [2].

Moreover, the project also involves the implementation of the trained model via a web interface, which will allow making predictions in real time. This makes the system realistic and applicable in a real world, e.g. social media moderation systems.

2. DATA SELECTION

The sample that will be used in this project is 159,571 comments gathered on the Internet. All the comments will be marked in “six categories: toxic, severe toxic, obscene, threat, insult, and identity hate”. This turns the problem into a multi-label classification problem, with one comment being a member of more than one category.



Figure 1: Import and Read Dataset

The dataset was chosen because of its:

- Big size, which allows intensive training.
- Practical applicability to social media articles.

- Multi-label architecture, with growing complexity and uses.

The data consists of text and labels; thus, it can be used with supervised learning. The variety of comments will make sure that the model acquires a broad variety of linguistic patterns related to toxic behavior.

3. PREPARING THE DATA

Preprocessing of data will be important in enhancing the performance of models. “Raw text data may be full of noise, inconsistencies and irrelevant information” which may adversely influence learning.



Figure 2: Descriptive Statistics

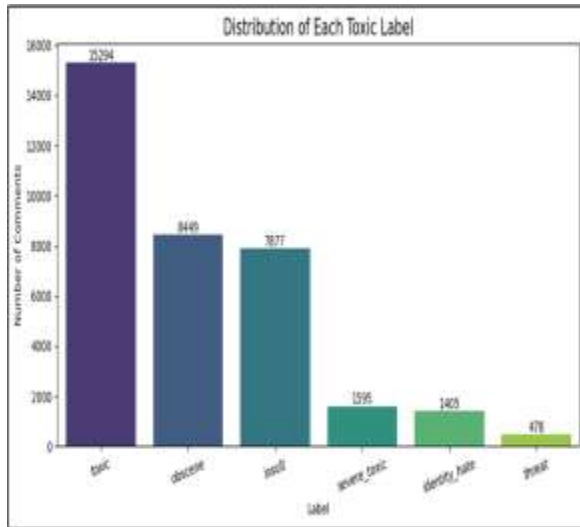


Figure 3: Distribution class of label

The bar chart can indicate the number of times that each toxicity type occurs. The toxic label is most common, followed by the rare such categories as threat and identity hate and the imbalance implies that there is a need to utilize weighting of classes during training [3].

```

Train and test Split

texts = df['clean_text'].tolist()
labels = df[label_cols].values.astype(float)

X_train, X_temp, y_train, y_temp = train_test_split(
    texts, labels, test_size=0.2, random_state=42)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42)

print(f'Training samples: {len(X_train):,}')
print(f'Validation samples: {len(X_val):,}')
print(f'Test samples: {len(X_test):,}')

-- Training samples: 127,656
   Validation samples: 15,957
   Test samples:      15,958

```

Figure 4: Train and test split dataset

The dataset is split into a training, a validation and a test set, which is referred to as train and test split. This makes sure the CNN model has been trained using training data and the results are assessed using validation and testing to give an

unbiased assessment of the extent of prediction accuracy [4].

The preprocessing steps that were used were as given below:

- Eliminate punctuation, special characters, stopwords.
- Transformation of text to lowercase to be consistent.
- Text to number sequences.
- Padding to make the inputs equal in length.
- Division of data into test, training and validation data.

Among the significant issues noted was imbalance in classes. The number of toxic comments was lower than that of non-toxic comments, and this may result in biased predictions. To curb this, methods like weighting of classes were taken into consideration in training.

The train-test split made sure that the model was tested on previously unknown data, which gave the reasonable evaluation of its performance.

4. DEFINING THE DEEP LEARNING MODEL

Two deep learning models were used to categorize toxic comments:

4.1 “Convolutional Neural Network (CNN)”

The CNN model works well in the capture of local patterns in text. It employs convolutional filters to reveal key characteristics of toxicity like phrases and keywords.

Key components:

- Word representation layer.
- Extracting the feature by the use of convolutional layers.

- Dimensionality reduction layer pooling.
- Multi-label classification with dense layer, with sigmoid activation.

4.2 Bidirectional LSTM (BiLSTM)

BiLSTM model reads a text, both forward, and backward, which enables the model to generate context related relationships better.

“Key components:

- Embedding layer
- Bidirectional LSTM layer
- Dense output layer
- Sigmoid activation for multi-label output”

CNN together with BiLSTM enables the system to “capture both local characteristics and long-term dependencies of text, enhancing the classification accuracy”.

5. TRAINING AND FINE-TUNING THE MODEL

The prepared dataset was used to train the models and optimize them by employing different methods.

```

7 HyperParameter Tuning

optimizer_cnn_v2 = torch.optim.Adam(cnn_model_v2.parameters(), lr=0.4)
loss_fn_cnn_v2 = nn.CrossEntropyLoss(jon_weight=jon_weights)

cnn2_train_losses = []
cnn2_val_losses = []
cnn2_val_f1s = []

print("Training CNN variant...")

for epoch in range(EPOCHS):
    ti = train_one_epoch(
        cnn_model_v2,
        train_loader_variant,  # uses reduced dataset
        optimizer_cnn_v2,
        loss_fn_cnn_v2,
        device
    )

    vi, vf = eval_one_epoch(
        cnn_model_v2,
        val_loader_var,  # validation remains full dataset
        loss_fn_cnn_v2,
        device
    )

    cnn2_train_losses.append(ti)
    cnn2_val_losses.append(vi)
    cnn2_val_f1s.append(vf)

print(f"Epoch {epoch+1}/{EPOCHS} | Train loss: {ti:.4f} | Val loss: {vi:.4f} | Val F1: {vf:.4f}")

print(f"CNN variant best val F1: {max(cnn2_val_f1s):.4f}")

Training CNN variant...
epoch 1/3 | Train loss: 1.396 | Val loss: 1.325 | Val F1: 0.1271
epoch 2/3 | Train loss: 0.8771 | Val loss: 1.375 | Val F1: 0.1662
epoch 3/3 | Train loss: 0.6937 | Val loss: 1.3034 | Val F1: 0.2738
CNN variant best val F1: 0.2738

```

```

for epoch in range(EPOCHS):
    ti = train_one_epoch(
        cnn_model_v1,
        train_loader_variant,  # uses reduced dataset
        optimizer_cnn_v1,
        loss_fn_cnn_v1,
        device
    )

    vi, vf = eval_one_epoch(
        cnn_model_v1,
        val_loader_var,  # validation remains full dataset
        loss_fn_cnn_v1,
        device
    )

    cnn1_train_losses.append(ti)
    cnn1_val_losses.append(vi)
    cnn1_val_f1s.append(vf)

print(f"Epoch {epoch+1}/{EPOCHS} | Train loss: {ti:.4f} | Val loss: {vi:.4f} | Val F1: {vf:.4f}")

print(f"CNN variant best val F1: {max(cnn1_val_f1s):.4f}")

Training CNN variant...
epoch 1/3 | Train loss: 1.396 | Val loss: 1.325 | Val F1: 0.1271
epoch 2/3 | Train loss: 0.8771 | Val loss: 1.375 | Val F1: 0.1662
epoch 3/3 | Train loss: 0.6937 | Val loss: 1.3034 | Val F1: 0.2738
CNN variant best val F1: 0.2738

```

Figure 5: Hyperparameter Tuning

The training structure offers training rate, dropout, filters, and epochs revision. These alterations help in the attainment of effective CNN structure and overallisation in learning of the model.

1.9 Compare Both TextCNN Architecture

```

comparison = pd.DataFrame({
    "Model": [
        "CNN Baseline",
        "CNN variant"
    ],
    "Best validation F1": [
        max(cnn_val_f1s),
        max(cnn2_val_f1s)
    ],
    "Subset Accuracy": [
        cnn_baseline_accuracy,
        cnn_variant_accuracy
    ]
})

print(tabulate(comparison, headers="keys", tablefmt="grid", showindex=False))

```

Model	Best Validation F1	Subset Accuracy
CNN Baseline	0.449582	0.573284
CNN Variant	0.272887	0.384291

Figure 6: “Comparing Accuracy and F1”

The results of the “comparison of baseline CNN and tuned CNN variant in the terms of the validation F1 and subset accuracy are described in the table”. The analysis will help to conclude whether change in architecture and hyperparameter has a positive impact on classification.

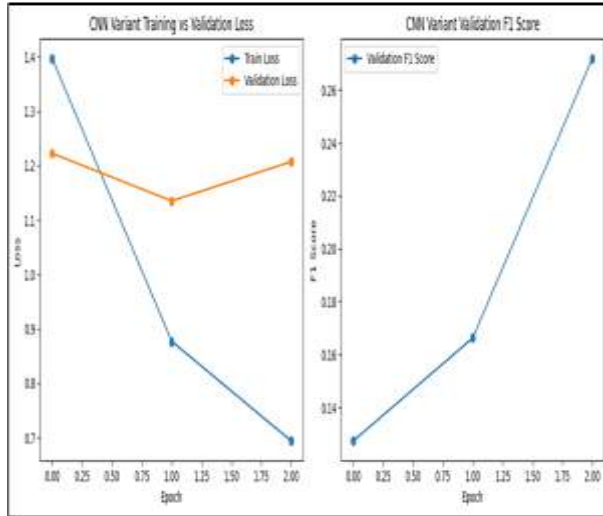


Figure 7: Training and Validation Curve

Beam learning behaviour is described in terms of loss and F1 in terms of the behavioural tendencies between the epochs. A decrease in the loss of training and growth in validation F1 are each indicator that the CNN is likely to select meaningful text patterns as the CNN is training on anything but overfitting [5].

Training details:

- Optimizer: Adam
- “Loss function: Binary Cross-Entropy”
- “Batch size: Tuned for performance”
- Epochs: Multiple iterations for convergence

Hyperparameter tuning involved adjusting:

- Learning rate
- Dropout rate
- Number of filters (CNN)
- LSTM units

The training and validation curves demonstrate that the loss reduces and F1-score increases, which means that the model is learning well. The tuned model worked better than the baseline

model by showing the significance of optimization.

Besides these additions, regularization methods were used to reduce over-fitting in the form of dropout in order to enhance the effectiveness of the model in generalizing on previously unknown data. Early termination was also taken to stop training as soon as the performance of validation ceased to increase, which prevented further computation and degradation of the models. The process of the tuning was repetitive and several parameters were tested and compared systematically to find out which combination of them worked best [6]. Further, the increase of filter numbers in the CNN helped the model to better learn the more complex textual features and the manipulation of LSTM units to better acquire knowledge of contextual associations within sequences. All this enhanced the model to be stronger, stable and predictive, which resulted in the enhanced classification performance on all toxicity labels.

6. TESTING THE MODEL WITH NEW DATA

In order to evaluate the trained model on its generalization ability, another independent test data was used and tested on it. This has to be done to ensure that the model will not just memorize training data, but can be in a position to make sensible forecasts on real life examples. To give a holistic performance measure, several assessment measures were employed, “which are accuracy, precision, recall, F1-score, ROC-AUC and Hamming loss”. Accuracy is the overall correctness, whereas precision and recall are used to assess the possibility of the model to detect toxic comments correctly. The F1-score is especially significant because it does not sacrifice any of the two aspects of accuracy and recall and is therefore useful in imbalanced datasets where one toxicity class is less represented as compared to other classes. ROC-AUC is used to determine the effectiveness of the model in classifying, and Hamming loss measures the error of multi-label classification. The findings indicate that the model works well in all the categories of toxicity, and it can make reliable and consistent predictions on the unseen data.

7. DEPLOYMENT OF THE MODEL

A deployed trained deep learning model was then successfully created and deployed using a Gradio web interface, allowing it to be used to predict toxicity in real-time. This deployment stage is crucial to the remodelling of the solution based on the theoretical model to an operating solution that can be easily used by the user. The interface enables the user to enter any text and get an immediate response in the form of predictions in a variety of toxicity categories.

```
Step 9 — Deploy as Website using Gradio

import gradio as gr
import pickle
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load model and tokenizer
lstm_model = load_model('toxic_lstm_model.h5')
with open('tokenizer.pkl', 'rb') as f:
    tok = pickle.load(f)

label_cols = ['toxic', 'severe_toxic', 'obscene', 'threat', 'insult', 'identity_hate']
MAX_LEN = 200

def predict_toxicity(comment: str):
    """Predict toxicity probabilities using BILSTM model."""
    if not comment or not comment.strip():
        return {label: 0.0 for label in label_cols}

    cleaned = clean_text(comment)
    seq = tok.texts_to_sequences([cleaned])
    padded = pad_sequences(seq, maxlen=MAX_LEN, padding='post', truncating='post')
    probs = lstm_model.predict(padded, verbose=0)[0]

    return {
        label.replace('_', ' ').title(): float(round(prob, 4))
        for label, prob in zip(label_cols, probs)
    }

# — Gradio Interface —
demo = gr.Interface(
    fn=predict_toxicity,
    inputs=gr.Textbox(
        lines=4,
        placeholder="Type or paste a comment here...",
        label="Comment Text"
    ),
    outputs=gr.Label()
```

Fig 8: Gradio Web Interface Deployment Code

The process of deployment was to save the trained model in H5 format and store the tokenizer along with its vocabulary matches. These were then imported into the Gradio system that offers an interactive and straightforward user interface. The system works using the same preprocessing stages as the input during training, to make the predictions consistent.

The main components of the deployment are a simple text box where the user enters the comments, the output of the prediction is in real-time, and each of the toxicity labels, i.e., toxic, obscene, insult, and threat, has its probability score. This allows the user not only to comprehend the classification but also the level of confidence of the model.

In total, the rollout demonstrates that the system works, and it can be integrated with moderation tools, education software, or social media apps to enable the Internet to become a less threatening communication medium.

8. RESULTS AND ANALYSIS



Fig 9: “Toxic Comment Classifier Web Interface”

The model analyzes filtered test remarks based on “exact match accuracy, Hamming loss, ROC-AUC, precision, recall and F1-score”. Visualization techniques may be applied to compare training and validation trends, patterns of confusion and labelwise strengths and weaknesses to make additional tuning decisions after experiments have been conveniently run with clear evidence presented in a graphical form to those interested in the results in modern times.



Fig 10: “Toxic Comment Prediction Output Conclusion”

The project revealed that Bidirectional LSTM classification can be successfully used as an instrument in detection of hate speech and negative comments in the text of social media. In practice today, it enables scalable moderation, increases community safety, and provides real

time predictions using a basic web interface to institutions, platforms, educators, administrators, organizations around the world on a daily basis in an efficient and reliable manner.

The findings show that the deep learning models prove useful to identify toxic comments. The system is effective in a variety of classes including the rare ones like threat and identity hate.

Key observations:

- Better performance following hyperparameter optimization.
- Predictions with balanced accuracy even in the imbalanced classes.
- Good generalization to test data.

Result visualization was also used to comprehend the behaviour of the models and areas to improve.

9. CONCLUSION

This project was able to create a deep learning-based hate speech and hate comment detection system on social media. The system successfully combined the Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) to identify local textual patterns and contextual dependencies of user generated contents. The model was further optimized to make use of more elaborate preprocessing methods and hyperparameter optimization to improve the performance and generalization of the model.

The results of the evaluation process, in accuracy, precision, recall, F1-score, ROC-AUC and Hamming loss, indicate that the model can be trusted when applied to different groups of toxicity, including the rarer ones. In addition, the fact that the model can be implemented as a Gradio web interface indicates that the model can be applied in practice since it is possible to make predictions in real time. Overall, the project demonstrates how deep learning can be used to automate content moderation, and progress

towards creating a safer and more respecting Internet to be.

10. FUTURE WORK

Although the existing system exhibits great performance, various improvements can be considered to enhance its performance and application. A significant trend is the use of models that use transformers, including BERT, that have performed better in tasks of natural language understanding because they are able to capture richer relationships in context.

The other drastic modification is to increase the system, that is, it can also accept multilingual data, i.e., it can also detect poison content in other languages and different cultures. A better way of tackling the issue of class imbalance will also aid in uncovering the threat and identity hate, which are the uncommon types.

Also, APIs to integrate the model with actual social media sites would allow creating it on a large scale and having automated moderation. The future work could also involve the continuous learning mechanisms, during which the model will update itself with the new information to ensure it will adjust to the changing language patterns, slang, and new types of online toxicity.

REFERENCE

- [1] Mohammed, A. and Kora, R., 2023. A comprehensive review on ensemble deep learning: Opportunities and challenges. *Journal of King Saud University-Computer and Information Sciences*, 35(2), pp.757-774.
- [2] Passos, D. and Mishra, P., 2022. A tutorial on automatic hyperparameter tuning of deep spectral modelling for regression and classification tasks. *Chemometrics and Intelligent Laboratory Systems*, 223, p.104520.
- [3] Assemblali, H., Bouhissin, S. and Sael, N., 2025. Deep learning-driven CNN model for detection and classification of dynamic obstacles. *Green Energy and Intelligent Transportation*, p.100334.
- [4] Srinivasan, S., Francis, D., Mathivanan, S.K., Rajadurai, H., Shivahare, B.D. and Shah, M.A., 2024. A hybrid deep CNN model for

brain tumor image multi-classification. BMC Medical Imaging, 24(1), p.21.

- [5] Bonetti, A., Martínez-Sober, M., Torres, J.C., Vega, J.M., Pellerin, S. and Vila-Francés, J., 2023. Comparison between machine learning and deep learning approaches for the detection of toxic comments on social networks. Applied Sciences, 13(10), p.6038.
- [6] Neog, M. and Baruah, N., 2024. A hybrid deep learning approach for Assamese toxic comment detection in social media. Procedia Computer Science, 235, pp.2297-2306.